

Important Questions

2 Marks – Question and Answers

1. What is overflow control in CSS? Give an example.

Answer:

Overflow control determines how content that exceeds an element's box is handled.

Example:

```
div {  
    overflow: auto;  
}
```

This adds scrollbars when content overflows.

2. What is a CSS Grid system?

Answer:

CSS Grid is a two-dimensional layout system that allows creation of rows and columns, enabling complex page layouts with simple CSS rules.

3. How do you create a basic grid layout in CSS?

Answer:

Use display: grid and define column/row sizes:

```
.container {  
    display: grid;  
    grid-template-columns: repeat(3, 1fr);  
}
```

4. What are CSS transition properties?

Answer:

Transition properties define how property changes are animated, including:

- transition-property
- transition-duration
- transition-timing-function
- transition-delay

5. What is JavaScript and why is it used?

- **Answer:** JavaScript is a lightweight, interpreted programming language used to create dynamic and interactive web pages. It works in the browser to handle user interactions, update the DOM, validate forms, and change page content dynamically.

6. Define primitive data types in JavaScript with two examples.

Answer: Primitive data types are immutable single-value types. Examples: **Number** (e.g., 42), **String** (e.g., "Hello"), **Boolean**, **Undefined**, **Null**, **Symbol**, **BigInt**.

7. What is a conditional statement? Write an example.

Answer: Conditional statements execute code based on true/false conditions. Example:

```
if (num > 0) {

    document.write("Positive");

}
```

8. What is the purpose of a switch statement?

Answer: A switch statement is used to execute one block of code among many alternatives based on matching values. It is often easier to read than multiple if-else statements.

9. What is the difference between for...in and for...of loops?

Answer:

- for...in iterates over **keys/indexes** of objects or arrays.
- for...of iterates over **values** of iterable objects like arrays or strings.

10. Define nodeType and nodeName with examples.

Answer:

- **nodeType** returns the numerical type of a node (e.g., 1 for element, 3 for text).
- **nodeName** returns the node's name (e.g., "DIV", "#text").
Example: element.nodeType → 1, element.nodeName → "P".

11. What is the use of setAttribute()?

Answer: setAttribute(name, value) is used to add or update an attribute of an HTML element, such as modifying title, class, or src.

12. Write any two DOM element selection methods.

1. `document.getElementById()` – selects an element by ID.
2. `document.querySelector()` – selects the first matching CSS selector element.

13. What is the purpose of an HTML form?

An HTML form is used to collect user input and send it to a server for processing.

Example:

```
<form action="submit.php">  
  
  <input type="text" name="username">  
  
</form>
```

14. What is a textarea element?

`<textarea>` allows users to enter multiple lines of text.

Example:

```
<textarea rows="4" cols="30">Enter your comment</textarea>
```

15. What is the purpose of the select element?

The `<select>` element creates a dropdown list for selecting predefined options.

Example:

```
<select>  
  
  <option>India</option>  
  
  <option>USA</option>  
  
</select>
```

16. What is an event listener?

An event listener waits for an event and runs a function when it occurs.

Example:

```
document.getElementById("btn").addEventListener("click", function(){
```

```
alert("Button pressed"); });
```

17. What are event object properties?

Event object properties give details about an event like target element, mouse position, key pressed.

Example:

```
document.addEventListener("click", function(e){  
  
    console.log(e.target); // element clicked  
  
});
```

18. What is asynchronous JavaScript?

Asynchronous JavaScript executes tasks without blocking other operations.

Example:

```
console.log("Start");  
  
setTimeout(()=>console.log("Async"), 1000);  
  
console.log("End");
```

19. What are scheduler functions? Give two examples.

Scheduler functions delay or repeat JS execution.

Examples:

- `setTimeout()` – runs once after delay
- `setInterval()` – runs repeatedly

```
setTimeout(()=>console.log("Delayed"), 2000);  
  
setInterval(()=>console.log("Repeat"), 1000);
```

20. What is the Fetch API used for?

Fetch API sends HTTP requests and handles responses asynchronously using promises.

Example:

```
fetch("data.json")  
  
.then(res => res.json())  
  
.then(data => console.log(data));
```

Descriptive Type

1. Explain CSS Overflow Control. Describe its different values with examples.

Answer:

The CSS **overflow** property controls what happens when content inside an element is too large to fit in its defined width/height. It determines whether the overflowing content should be **visible**, **hidden**, **scrollable**, or shown with scrollbars only when needed.

Types of Overflow:

1. overflow: visible (default)

Content will flow outside the element's box and remain visible.

```
.box1 {  
  width: 200px;  
  height: 80px;  
  background: coral;  
  overflow: visible;  
}
```

2. overflow: hidden

Extra content will be clipped (cut off).

```
.box2 {  
  width: 200px;  
  height: 80px;  
  background: lightgreen;  
  overflow: hidden;  
}
```

3. overflow: scroll

Always shows both scrollbars (even if not needed).

```
.box3 {  
  width: 200px;  
  height: 80px;  
  background: lightblue;  
  overflow: scroll;  
}
```

4. **overflow: auto**

Scrollbars appear **only if** content overflows.

```
.box4 {  
  width: 200px;  
  height: 80px;  
  background: yellow;  
  overflow: auto;  
}
```

Example HTML:

```
<div class="box1">This is long content...</div>  
<div class="box2">This is long content...</div>  
<div class="box3">This is long content...</div>  
<div class="box4">This is long content...</div>
```

2. Explain CSS Grid Layout and describe how grid-template-rows, grid-template-columns, and gap work.

Answer:

CSS Grid is a **two-dimensional layout system** that organizes content into **rows and columns**. It provides full control over spacing, sizing, and alignment.

1. grid-template-columns

Defines the number and width of columns.

```
grid-template-columns: 1fr 2fr 1fr;
```

Creates **three columns**, where the middle one is twice as wide.

2. grid-template-rows

Defines the number and height of rows.

```
grid-template-rows: 100px auto 50px;
```

Creates 3 rows: fixed, flexible, and fixed.

3. gap

Sets spacing between rows and columns.

```
gap: 20px;
```

Example:

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 100px 200px;  
  gap: 10px;  
}  
  
.item {  
  background: steelblue;  
  color: white;  
  padding: 20px;  
}
```

Html code

```
<div class="container">  
  <div class="item">A</div>  
  <div class="item">B</div>  
  <div class="item">C</div>  
  <div class="item">D</div>  
  <div class="item">E</div>  
  <div class="item">F</div>  
</div>
```

3. Explain CSS Transition Properties. List all properties and give a suitable example.

Answer:

CSS **transitions** allow smooth changes between two states (normal → hover/active). A transition occurs only when the property value changes.

Transition Properties:

Property	Description
transition-property	CSS property to animate
transition-duration	Time animation takes
transition-timing-function	Speed curve (ease, linear, ease-in)
transition-delay	Delay before animation starts

transition: property duration timing-function delay;

```
.box {  
  width: 100px;  
  height: 100px;  
  background: blue;  
  transition: width 1s ease-in-out, background-color 1s;  
}
```

```
.box:hover {  
  width: 200px;  
  background: green;  
}
```

Explanation:

When hovered, the box will smoothly expand in width and change color.

4. Explain different CSS Transform Functions with examples.

Answer:

The **transform** property modifies an element's shape, size, and position **without affecting surrounding elements**.

1. translate(): Move element

```
transform: translate(50px, 20px);
```

2. scale(): Resize element

```
transform: scale(1.5);    /* 150% bigger */
```

3. rotate(): Rotate element

```
transform: rotate(45deg);
```

4. skew(): Tilt element

```
transform: skewX(30deg);
```

5. Multiple transforms

```
transform: rotate(30deg) scale(1.2) translateX(50px);
```

Example with effect:

```
.box {  
    width: 100px;  
    height: 100px;  
    background: tomato;  
    transition: 0.5s;  
}  
  
.box:hover {  
    transform: rotate(45deg) scale(1.2);  
}
```

Explanation:

On hover, the box rotates and increases in size smoothly due to transition.

5. Explain JavaScript Data Types with examples.

Answer:

JavaScript supports **primitive** and **non-primitive** data types. Primitive types include: **Number, String, Boolean, Undefined, Null, Symbol, BigInt**. Non-primitive types include: **Object, Array, Function, Date, Regex**. Primitive values are immutable, while non-primitive types store collections.

Example:

```
let age = 25;      // Number
let name = "Husna"; // String
let bool = true;   // Boolean
let data = {name:"Amit", age:25}; // Object
let colors = ["red","green"]; // Array
```

Data types decide **what operations** can be performed and **how memory is allocated**.

6. Compare var, let, and const with examples.

Answer:

Feature	var	let	const
Scope	Function-scoped	Block-scoped	Block-scoped
Redeclaration	Allowed	Not allowed	Not allowed
Reassignment	Allowed	Allowed	Not allowed

Example

```
var x = 10;
var x = 20; // allowed
```

```
let y = 10;
// let y = 20; // error (no redeclare)
y = 20; // allowed
```

```
const z = 100;
// z = 200; // error
```

Use **let/const** for modern JavaScript to avoid accidental redeclarations.

7. Explain Arithmetic, Comparison, and Logical Operators with examples.

Answer:

JavaScript provides:

Arithmetic Operators

+ - * / % ++ --

Example:

```
let sum = 10 + 5; // 15
```

Comparison Operators

== === != !== > < >= <=

```
10 == "10"; // true
```

```
10 === "10"; // false
```

Logical Operators

&& || !

Example:

```
true && false; // false
```

Operators help in **performing calculations, decision making, and conditional execution.**

8. Describe different Conditional Statements with examples.

Answer:

JavaScript supports:

- **if**
- **if-else**
- **else if**
- **switch**
- **ternary operator**

Example (if-else):

```
let num = -5;
```

```
if (num > 0) console.log("Positive");
```

```
else if (num < 0) console.log("Negative");  
else console.log("Zero");
```

Example (switch):

```
switch(day){  
  case 1: console.log("Sunday"); break;  
  default: console.log("Invalid");  
}  
  
switch(day){  
  case 1: console.log("Sunday"); break;  
  default: console.log("Invalid");  
}
```

Conditions decide the **flow of the program** based on truth values.

9. Explain different types of loops in JavaScript with examples.

Answer:

JavaScript loops:

- **for** – when number of iterations is known
- **while** – condition checked before loop
- **do...while** – runs at least once
- **for...in** – iterates object keys
- **for...of** – iterates values of iterable

Example (for loop):

```
for(let i=1; i<=5; i++){  
  console.log(i);  
}
```

Example (for...of):

```
for(let color of ["red","blue"])  
  console.log(color);
```

Loops help in **repetition of tasks** until a condition is satisfied.

10. What are JavaScript Functions? Explain parameters, arguments, and return values with an example.

Answer:

A **function** is a reusable block of code that performs a task.

- **Parameters** → placeholders in function definition
- **Arguments** → actual values passed during call
- **Return** → sends back a value

Example:

```
function sum(a, b){ // parameters
  return a + b;
}
```

```
let result = sum(5, 3); // arguments
```

Functions increase **reusability**, **modularity**, and **readability**.

11. What is DOM? Explain DOM hierarchy with an example.

Answer:

The **Document Object Model (DOM)** is a tree-like representation of an HTML document. It allows JavaScript to **access**, **modify**, and **update** elements dynamically.

Hierarchy Example:

```
document
├── html
│   ├── head
│   └── body
│       ├── h1
│       └── p
```

DOM enables dynamic updates, event handling, and interactive webpages.

12. Explain element selection methods with examples.

Answer:

1. `getElementById()`

Selects element by ID

```
document.getElementById("demo").innerHTML = "Hello";
```

2. `getElementsByClassName()`

Returns HTMLCollection

```
document.getElementsByClassName("myClass")[0].style.color="blue";
```

3. `querySelector()`

Returns first matching element

```
document.querySelector(".myClass");
```

4. `querySelectorAll()`

Returns NodeList

```
document.querySelectorAll("p");
```

These methods allow JavaScript to **find and manipulate HTML elements**.

13. Explain HTML Form Structure with an example.

Answer:

A form in HTML is used to collect user input and send it to a server for processing. It is created using the `<form>` tag, which supports attributes like:

- **action** → URL where data is sent
- **method** → GET or POST
- **enctype** → encoding type for data
- **autocomplete** → on/off

A form contains controls like `<input>`, `<textarea>`, `<select>`, buttons, etc.

Example:

```
<form action="/submit" method="POST">
```

```
<label>Name:</label>
```

```
<input type="text" name="username">
```

```
<label>Message:</label>
```

```
<textarea name="msg"></textarea>
```

```
<button type="submit">Submit</button>
```

```
</form>
```

This form contains structured input components and sends data to the server using **POST**. Forms are essential for login pages, registrations, and data submissions.

14. Describe different input element types with examples.

Answer:

HTML supports many input types for collecting different kinds of data:

Common Input Types

- text
- password
- email
- number
- radio
- checkbox
- date
- file
- submit

Example:

```
<input type="text" placeholder="Enter name">
```

```
<input type="email" placeholder="Enter email">
```

```
<input type="number" min="1" max="10">
```

```
<input type="radio" name="gender" value="Male"> Male
```

```
<input type="checkbox" value="Agree"> I Agree
```

Explanation:

Each input type enforces specific validation and user interface, helping ensure proper data collection.

15. Explain Textarea and Select form controls with examples.

Answer:

Textarea

Used to collect multi-line text.

```
<textarea rows="4" cols="30">Enter your message...</textarea>
```

Select

Provides dropdown options using <select> and <option>.

```
<select>
```

```
  <option value="india">India</option>
```

```
  <option value="usa">USA</option>
```

```
</select>
```

Explanation:

- **Textarea** is ideal for comments, descriptions, feedback.
- **Select** ensures users choose from predefined values, improving accuracy.

16. What is Form Validation? Explain client-side validation with an example.

Answer:

Form validation ensures that user inputs are correct before submission.

Types of Validation

- **Client-side (JavaScript / HTML5)**
- **Server-side**

Example (HTML5 validation):

```
<input type="email" required>
```

Example (JavaScript validation):

```
function validate() {
```

```
let name = document.getElementById("name").value;

if (name === "") {

    alert("Name is required!");

    return false;

}

return true;

}
```

Explanation:

Validation prevents incorrect or incomplete data from being sent, ensuring data quality and enhancing security.

17. Explain the Event Model in JavaScript with an example.

Answer:

The Event Model defines how events (click, keyup, load) are handled in the browser.

Components:

1. Event Target → element receiving event
2. Event Listener → function that responds
3. Event Object → information about event (type, key, target)

Example:

```
document.getElementById("btn").onclick = function () {

    alert("Button clicked!");

};
```

Explanation:

The event model is foundational for interactive pages where JavaScript reacts to user actions.

18. What is Event Propagation? Explain capturing and bubbling with examples.

Answer:

Event propagation describes how events travel through the DOM.

Phases:

1. **Capturing Phase** → event travels from root to target
2. **Target Phase** → event reaches actual element
3. **Bubbling Phase** → event travels back to root

```
document.getElementById("outer").addEventListener("click", () => {  
  
    alert("Outer Div");  
  
}, true); // capturing  
  
document.getElementById("inner").addEventListener("click", () => {  
  
    alert("Inner Div");  
  
}, false); // bubbling
```

Explanation:

Capturing handles events from outer → inner, bubbling from inner → outer.
Understanding propagation prevents accidental event triggering.

19. Explain Event Object Properties with examples.

Answer:

The event object contains details about the event.

Common Properties:

- event.type
- event.target
- event.clientX, event.clientY
- event.key (for keyboard events)

Example:

```
document.addEventListener("click", function (event) {  
  
    console.log("Clicked element:", event.target);  
  
    console.log("Event type:", event.type);  
  
});
```

Explanation:

The event object provides context required to handle events intelligently, e.g., checking which element was clicked.

20. Explain Fetch API with JSON parsing and serialization with examples.

Answer:

Fetch API is used to make HTTP requests and receive data from a server.

Example: GET request

```
fetch("https://api.example.com/data")  
  
  .then(res => res.json()) // JSON parsing  
  
  .then(data => console.log(data));
```

JSON Serialization (object → JSON string):

```
let obj = { name: "John", age: 25 };  
  
let json = JSON.stringify(obj);
```

JSON Parsing (JSON string → object):

```
let parsed = JSON.parse(json);
```

Explanation:

- Fetch simplifies HTTP communication
- JSON is lightweight data format
- `JSON.parse()` and `JSON.stringify()` allow conversion between objects and text for APIs